Recitation Assignment #2

October 2, 2024

Today we're going to do an actual simulation of a very simple physical system. You may complete this in class. However, if you are unable to do so, it is expected that you complete this for recitation next time. We'll make a model of projectile motion: a ball thrown through the air.

We will use the following new concepts:

Programming:

- Vector math
- "While" loops
- "If Statements
- Outputting simple results

Physics:

- A particle moving under constant acceleration
  1. $\Delta\vec{v} = \vec{a}\Delta t$
  2. $\Delta\vec{r} = \vec{v}\Delta t$
- Elastic bounces

Each of these concepts will be important in later programs, so make sure you master them here. You will find it very useful to have an open copy of the the vpython help pages (linked on the recitation page of the class website), while you work.

1. Open up a new jupyter notebook
2. Write the following starter program inside a cell of your new notebook. Be sure to run it after every few lines to be sure you've transcribed it correctly:

```
#our starter code!

L = 10
```

```
H = 0.2

Xaxis=box(pos=vector(L/2.,-H/2.,0), length=L, height=H, width=L/3.,
texture=textures.wood)

#these next couple of lines point the "camera" inside of vpython

scene.autoscale=0

scene.center=vector(L/2., 0.4*L, 0)

scene.forward=vector(0, -0.2, -0.9)

scene.scale=0.7*L
```

A few notes:

- While I did not include it in the sample, your code should include "from vpython import *" at the top, which imports all of the vpython functions you'll be using.
- The first line is called a comment. It's meant to be readable by humans. You'll want to use comments a lot. Just start a line with # and everything after that will be ignored by python.
- The 'box' line is quite long. If any line becomes too long to read in your window, there is a method to continue code on the following line. Simply type a backslash '\' at the point where you want to continue on the next line, and it will work the same way. For example:

  ```
  Xaxis=box(pos=vector(L/2.,-H/2.,0), length=L, height=H, width=L/3., \
  texture=textures.wood)
  ```

  the backslash at the end of the first line tells python to read the next line as part of the first.

- I defined a few numbers, L and H that get used a lot in the program. If I later want to make everything bigger, I can just set those numbers.
- The **box** function should be straightforward, but I have put a lot of extra things in there.
- All of those **scene** commands may look confusing, but they fix the camera. Vpython is otherwise set up to expand the field of view if, for instance, the ball leaves the original area.

3. Now it's time to add the ball. Create a sphere called "ball" and put it at the origin. To do this, you will want to define a few parameters.

In particular, include the following line near the top of your code:

```
R=0.3
```

And give your ball a radius of R. You may make your ball have whatever color or materials you like.

4. You may notice that your ball appears to be embedded in the floor. That's no good! Figure out an algebraic way to set the position of your ball so that it's just resting on the floor.

As a hint, you can set the y-position of the ball all by itself by saying something like:

```
ball.pos.y=4.0
```

(only you obviously won't want to set it to 4)

5. We need to set several numbers to be used in the program. I am going to give them names I like. You should be aware that you can call them whatever you want and the program will still work. However, they need to be **declared** before you use them!

I've suggested a place to put numbers like this near the top of your code.

```
#Initialize my parameters
dt=0.01
tmax=10.
t=0.
g=9.8
v0=8.
theta=pi/3
```

All I've done is define 6 numbers (and once again provided a comment to help myself out later when I look at the code). What are they going to stand for? dt will represent the amount of time between frames of our simulation. tmax tells the simulation how long to run. t = 0 tells the program what time to start at. g = 9.8 will be used to put gravity into the code.

I've also put in a speed v0 that stands for 8 m/s (the initial speed of the ball) and theta, the initial trajectory of the ball.

The computer doesn't know anything about units, so I need to be very careful. All of my units are in MKS so they make sense. If you mix and match (with pounds and miles and lightyears and so forth), the computer will not give you any sort of warning.

Run this code. Has anything changed?

6. Part of the issue is that while we've given numbers indicating the speed and direction of the ball, we haven't set its velocity. To do that, we'd like to define the number:

```
ball.v=vector(v0*cos(theta),v0*sin(theta),0)
```

This must be done *after* you've set up the ball in the first place

But does this mean that there is physics? No. The computer has no idea what "v" means.

Run the code. Nothing happens. We're going to need to tell the computer how to add some physics.

7. All programming languages have the concept of a "while" loop. This is basically a way of telling the computer that it should run a set of commands again and again until a specified time. This is incredibly important, because the advantage of a computer is that it can run lots and lots of commands.

Write the following (using the Tab key to indent).

```
while (t < tmax):
    t=t+dt
    print(t)
print("Now I am done.")
```

You will note that in each step, the clock advances 0.01 until it hits 10, and then it tells you that it's done.

But wait! This loop was much faster than 10 seconds, wasn't it? If you want the clock to run in real time, you need the "rate" command, which will tell the loop the maximum number of times to run each second. Add:

```
rate(1./dt)
```

to your code right under the start of the while loop (but indented). Think about it: if dt=0.01, then 1./dt=100, or the loop should go 100 times per second, which is what we want!

8. Time to add some physics. The simplest physics we've seen is that a particle in the absence of gravity (which we haven't added yet) will simply move in a straight line. We've done a one-line version of this in the last class. Under the loop, add the line:

```
ball.pos=ball.pos+ball.v*dt
```

Mathematically, this is basically saying that at each time step:

$$\Delta\vec{r} = \vec{v}\Delta t$$

When you run this, you'll see that we have a movie!

Of course, this system doesn't have gravity.

9. To add gravity, we need to add an additional command under the loop. We'll need to update the velocity as well. The acceleration is a vector:

$$\vec{a} = -g\hat{\jmath}$$

Where $\hat{\jmath}$ is simply a unit vector along the y-axis (up/down). This can be written, in python, as:

```
a = vector(0, -g, 0)
```

or:

```
a = -g * vector(0, 1, 0)
```

Define this *outside* the while loop. You don't want to do the same calculation again and again if you're always going to get the same number. Then, inside the loop, write a code representing:

$$\Delta\vec{v} = \vec{a}\Delta t$$

Use your position updates as an example.

Run the code. You'll notice that there is still some physics missing. The ball goes straight through the floor!

10. Also, it's easy to lose track of the ball. We may want to add a "trail" to indicate where it's been. Above the loop, add the line:

```
trail=curve(color=color.red)
```

This does nothing initially. However, every time you execute the command:

```
trail.append(ball.pos)
```

A new line segment is added, terminating at the current position of the ball, creating a continuous curve. Add the above line into your 'while' loop.

11. As a first step toward bouncing the ball, we need to figure out when it strikes the floor. This involves an "if" statement. In particular, after we update the position and velocity of the ball in every timestep, we want to check:

```
if ((ball.pos.y < ball.radius) & (ball.v.y  < 0)):
    print(ball.pos.x)
    #bounce! (remember to put in physics)
```

There is *a lot* here. The "if" statement is basically checking two things:

1) Whether the ball is below the ground. Notice that since the position of the ball is defined by the center, for the ball to hit the floor, the y-coordinate just has to be less than the radius.

2) Whether the ball is on the way down.

It checks both of these things, and only prints out the x coordinate of the ball if both are true. You may also want to delete the "print(t)" command, since that's just making things messy.

12. Finally, we're going to want to make the ball bounce. The trick is that if we find that the ball is below the floor, we simply want to reverse the y-component of the velocity. In other words:

```
ball.v.y = -ball.v.y
```

13. Your program is now complete, and you should send it to your TA, but if you've done some high school physics (which most of you have), you'll recall a bit about projectile motion.

A ball launched from the ground with initial velocity $\vec{v}$ will have a vertical position:

$$y(t) = v_{y0}t - \frac{1}{2}gt^2$$

Which, using algebra means that it will hit the ground again (y=0) when

$$t_{land} = \frac{2v_{y0}}{g}$$

And thus, it will cover a distance of

$$x = v_{x0}t_{land} = \frac{2v_{x0}v_{y0}}{g}$$

Or equivalently

$$x = \frac{2{v_0}^2}{g} sin\theta \ cos\theta$$

Do your answers match your expectations? Why or why not? Does the finite size of the ball come into play? What happens if you adjust the timesteps (dt=0.1, dt=0.5, dt=0.001)?

14. **Extra Credit!:** We will give you extra credit (maximum of +10%) if you can have the ball correctly bounce on the table and then fall off when there is no more table to bounce on.

Make sure you test your code and submit it to Kendall (via email) by next week's recitation.